Process & Decision Documentation

Project / Assignment Decisions

During development, one key decision was simplifying the interaction system to ensure the platform mechanics worked reliably. Early versions of the sketch had issues where landing detection and visual feedback were inconsistent, which caused sound and particle effects to trigger at incorrect times. I adjusted the landing detection logic and simplified some conditions so that feedback events (landing sound, particles, and collision sparks) would trigger consistently.

Another design decision was adding a simple goal structure to the sketch. Coins were introduced as collectible objects and an exit door appears after all coins are collected. This change provided a clearer objective for the player and helped structure the gameplay loop of movement, collection, and level completion.

Role-Based Process Evidence

Entry Header

Name:
Kiki Tan

Role(s):
Game designer and programmer

Primary responsibility for this work:
Designing and implementing the platformer mechanics, player interactions, feedback effects, and game flow.

Goal of Work Session

The goal of this work session was to extend the basic sprite animation example into a small interactive platformer. This included implementing player movement, jumping mechanics, coin collection, attack interactions, and visual and sound feedback for collisions and actions. Additional work focused on debugging interaction systems and ensuring the game loop functioned reliably.

Tools, Resources, or Inputs Used

- p5.js library and reference documentation
- Course lecture examples and side quest starter code
- ChatGPT (GenAI) for debugging assistance and implementation guidance
- Iterative testing within the p5.js editor

GenAI Documentation

Date Used:
March 5, 2026

Tool Disclosure:
ChatGPT (GPT-5 series model)

Purpose of Use:
GenAI was used primarily for debugging assistance and for guidance on implementing particle effects, collision detection adjustments, and interaction logic.

Summary of Interaction:
The tool helped identify issues related to landing detection, collision feedback triggers, and particle effect timing. It also provided suggestions for restructuring parts of the code to ensure visual and sound feedback triggered more reliably.

Human Decision Point(s):
Several suggested implementations were modified during development. For example, the landing detection logic was adjusted after testing showed that some suggested conditions triggered effects inconsistently. I simplified the logic and repositioned certain UI elements, such as the coin counter, based on visual testing.

Integrity & Verification Note:
All GenAI suggestions were tested manually within the p5.js environment. Code suggestions were only used if they functioned correctly and matched the intended gameplay behavior.

Scope of GenAI Use:
GenAI did not design the gameplay concept, overall structure of the sketch, or the game objective. These decisions were made independently and implemented through manual coding and testing.

Limitations or Misfires:
Some early suggestions produced unintended behavior, such as delayed landing effects or disappearing elements after collisions. These issues required manual debugging and adjustment.

Summary of Process (Human + Tool)

Development proceeded iteratively through testing and debugging cycles. Initial implementation focused on movement and physics behavior for the player character. After basic movement worked, coins and collection logic were added to introduce a simple objective.

Later iterations added feedback elements such as sound effects, spark particles, and collision responses. Several debugging steps were required to ensure landing events triggered consistently and that UI elements such as the coin counter remained visible. Through repeated testing and adjustment, the interaction systems were stabilized and integrated into a functioning gameplay loop.

Decision Points & Trade-offs

One key decision involved simplifying the physics interaction conditions rather than implementing a more complex system. While a more detailed physics system could allow additional mechanics, simplifying the logic ensured the game remained stable and easier to debug.

Another trade-off was limiting the level complexity. Instead of building multiple levels or obstacles, the sketch focuses on a small environment with coins and an exit door. This allowed development time to focus on interaction feedback and responsive controls.

Verification & Judgement

Testing was conducted directly in the p5.js runtime environment. Each interaction (movement, jump, attack, coin collection, door interaction) was tested repeatedly to confirm that the expected visual and audio feedback occurred. Adjustments were made whenever interactions behaved inconsistently.

Assignment instructions and example sketches from the course were also referenced to ensure the sketch met the expectations for the side quest.

Limitations, Dead Ends, or Open Questions

Some timing issues with sound effects and landing detection required multiple debugging attempts before functioning consistently. The sketch currently includes a small level structure, and the gameplay could be expanded further with additional obstacles or platform layouts.

Future improvements could include smoother animation transitions, more complex enemy interactions, or expanded level design.

Appendix

GenAI Interaction Transcript

User:
I'm working on a small platformer sketch in p5.js. The player can move and jump, but I'm running into a strange issue. After the character lands from a jump, all the objects suddenly disappear and only the background remains. I'm not sure what is causing this.

AI:

Thanks for the detailed description. Issues where **game objects disappear after a specific event (such as landing)** are usually related to state changes or logic that unintentionally resets arrays or sprite objects.

Let's go through a few **common causes and debugging steps**.

Possible Causes

The most likely reasons for this type of issue are:

• **Arrays being reset during the draw loop**
• **Sprites being removed unintentionally after collision events**
• **Game state variables triggering a reset condition**
• **Incorrect condition checks tied to landing detection**

Because the problem happens **specifically when landing**, it's possible that some logic connected to ground detection is modifying the game state.

Things to Check First

You can quickly narrow the problem by checking the following:

1. Look for array resets

Search your code for patterns like:

- `array = []`
- `sprites.remove()`

- `allSprites.remove()`

If any of these occur inside `draw()` or inside a collision condition, they may accidentally clear objects.

---

2. Verify landing logic

Landing detection usually relies on conditions such as:

- velocity becoming positive or zero
- collision with ground
- a grounded state variable

If a landing check is written like this:

```
if (player.collides(ground)) {
   resetLevel();
}
```

then every time the player touches the ground the level could reset.

---

3. Check for unintended reset functions

Sometimes projects include functions like:

- `resetLevel()`
- `restartGame()`
- `initializeObjects()`

Make sure these are **not being triggered during landing events**.

Helpful Debugging Strategy 🔍

A simple way to track what is happening is to temporarily log key values.

For example:

```
console.log("Player velocity:", player.vel.y)
console.log("Ground collision:", player.colliding(ground))
```

Watching these values will help you see whether a condition is firing unexpectedly.

Landing Detection Logic (Typical Pattern)

A reliable landing detection pattern looks like this:

```
if (!wasGrounded && isGrounded) {
   // player just landed
}
```

Where:

- `wasGrounded` = grounded state from the previous frame
- `isGrounded` = grounded state in the current frame

This prevents logic from firing every frame.

Recommended Debugging Steps

Try the following in order:

- Temporarily comment out any reset or remove functions
- Print debug logs for collision checks
- Track whether the grounded state changes when landing
- Verify that sprite arrays are not being cleared unintentionally

These steps should help you locate exactly where the objects are disappearing.

Tip:
When debugging physics or collision systems, isolating **state transitions** (like airborne → grounded) often reveals where logic is triggering unexpectedly.

Let me know what you find after checking those conditions.

User:
I checked and it seems like the issue happens right when the landing condition triggers. I had a section where I tried to detect landing, and that might be connected.

AI:

That's a useful discovery 👍
If the issue occurs exactly when the **landing condition triggers**, the problem is likely related to how that condition is structured.

Let's look at a few implementation details that commonly cause this behavior.

Common Landing Detection Pitfalls

1. Landing logic placed in the wrong location

If landing detection is inside a loop that updates other objects, it may accidentally affect them.

For example:

```
for (let i = 0; i < coins.length; i++) {
   if (landingCondition) {
      coins = [];
   }
}
```

This would clear the entire coin array when landing.

---

2. Reset logic tied to physics events

Sometimes code that was meant to trigger once becomes tied to a condition that runs every frame.

Example:

```
if (player.vel.y === 0) {
   resetObjects();
}
```

Because the player often has velocity `0` when standing still, this may repeatedly run.

---

3. Incorrect state tracking

Landing detection works best when you track **previous state vs current state**.

Example structure:

```
let wasGrounded = false;

function update() {

   let isGrounded = player.colliding(ground);

   if (!wasGrounded && isGrounded) {
      // landing event
   }

   wasGrounded = isGrounded;
}
```

This ensures the landing logic only runs once.

Implementation Tip 🧩

If you are using particle effects or sounds on landing, try keeping that logic separate from gameplay state.

Example structure:

```
if (!wasGrounded && isGrounded) {

   spawnLandingParticles();
   playLandingSound();

}
```

Avoid placing **game resets or object removal inside this block**.

Quick Diagnostic Trick

You can also temporarily disable the landing code entirely:

```
if (false && landingCondition) {
   // landing code
}
```

If objects stop disappearing, then the landing block is confirmed as the source.

User:
I adjusted the landing detection logic and removed a reset that was tied to that condition. Now the objects stay visible after landing.

AI:

Great — that confirms the issue was related to the landing condition triggering unintended behavior.

At this stage it sounds like the core movement system is working correctly.

As you continue building the sketch, keep these principles in mind:

Key Implementation Practices

• Keep **physics events** separate from **game state resets**
• Track **previous frame vs current frame state** for transitions
• Avoid clearing arrays or sprites inside collision loops
• Use small debug logs when unexpected behavior occurs

These practices help keep interaction systems stable as the project grows.

User:
Now that the main movement works, I want to add some small spark effects when the player hits a wall or attacks. I already decided to use sparks as a visual feedback effect. I'm not sure what the best way to implement the particles would be.

AI:

That is a good use case for a **simple particle system**.
In small game sketches, spark effects are typically implemented using a lightweight structure where each particle stores position, velocity, and lifespan.

Below is a typical implementation approach.

Basic Particle Data Structure

Each particle can be represented as a small object with a few properties:

```
{
   x: number,
   y: number,
   vx: number,
   vy: number,
   life: number
}
```

Where:

- `x`, `y` = particle position
- `vx`, `vy` = velocity vector
- `life` = remaining frames before removal

Particles are usually stored inside an array.

Example:

```
let particles = [];
```

Particle Creation

When a collision or attack occurs, you can push multiple particles into the array.

Typical structure:

```
function spawnSparks(x, y) {

   for (let i = 0; i < 10; i++) {

      particles.push({
         x: x,
         y: y,
         vx: random(-2,2),
         vy: random(-3,-1),
         life: 20
      });

   }

}
```

Important details:

- Random velocity values help create a natural spread
- Slight upward motion (`vy` negative) creates the spark burst effect

---

Particle Update Loop

Particles must be updated each frame inside the main draw loop.

Typical steps include:

1. Update position

```
p.x += p.vx
p.y += p.vy
```

2. Apply gravity

```
p.vy += 0.1
```

3. Reduce lifespan

```
p.life -= 1
```

Particles are removed when lifespan reaches zero.

Particle Removal

Removing expired particles prevents the array from growing indefinitely.

A common pattern is iterating **backwards** through the array.

Example:

```
for (let i = particles.length - 1; i >= 0; i--) {

   let p = particles[i];

   if (p.life <= 0) {
      particles.splice(i,1);
   }

}
```

Iterating backwards prevents index shifting issues when removing items.

Drawing the Sparks ✨

Spark particles are typically drawn as small circles or rectangles.

Example:

```
circle(p.x, p.y, 3);
```

You may also vary brightness or transparency based on lifespan.

Example idea:

• early life → bright
• late life → faded

Example pattern:

```
let alpha = map(p.life,0,20,0,255);
fill(255,200,80,alpha);
```

Triggering the Spark Effect

Your spark spawn function should be called **only when collisions occur**.

Common trigger locations include:

• player hitting a wall
• attack animation frame
• weapon contact

Example:

```
if (player.colliding(wallL) || player.colliding(wallR)) {

   spawnSparks(player.x, player.y);

}
```

For attacks:

```
if (attackTriggered) {

   spawnSparks(player.x + offset, player.y);

}
```

Implementation Notes ⚙

When implementing particle systems, a few things help keep the system stable:

Avoid resetting the particle array

Particles should only be removed individually when their lifespan expires.

Keep particle counts moderate

For small sketches:

• 8–15 particles per burst
• lifespan around 10–25 frames

This keeps performance stable.

Update particles every frame

Particles must be both **updated and drawn** every frame.
If update logic runs but drawing does not, particles will appear to disappear instantly.

Minimal Workflow Summary

A simple spark particle system generally requires four steps:

1 Create particle objects
2 Store them in an array
3 Update position and lifespan each frame
4 Remove expired particles

User:
I implemented the particle array and the sparks appear when the player hits the wall. The attack sparks also work now.

AI:

Good — that indicates the particle system is functioning correctly.

As long as:

• particles update every frame
• expired particles are removed
• spawn events are tied to collision conditions

the effect should remain stable as additional interactions are added.

If you later add additional particle types, you can reuse the same structure and adjust parameters such as velocity range or lifespan.

User:
Now that the main movement works, I want to add some small spark effects when the player hits a wall or attacks. I already decided to use sparks as a visual feedback effect. I'm not sure what the best way to implement the particles would be.

AI:

Great — a **lightweight particle system** is usually the simplest way to implement spark effects in a p5.js sketch. The idea is to treat each spark as a small object with a short lifespan and update it every frame.

Below is a typical implementation structure.

Basic Particle Structure

Each spark particle can be represented with a few properties:

- position (`x`, `y`)
- velocity (`vx`, `vy`)
- lifespan (`life`)

Example structure:

```
{
  x: number,
  y: number,
  vx: number,
  vy: number,
  life: number
}
```

Particles are usually stored in an array.

Example:

```
let particles = [];
```

This array will hold all currently active spark particles.

Creating Sparks

When the player hits a wall or performs an attack, you can generate several particles at once.

Typical pattern:

```
function spawnSparks(x, y) {

  for (let i = 0; i < 10; i++) {

    particles.push({
      x: x,
      y: y,
      vx: random(-2, 2),
      vy: random(-3, -1),
      life: 20
    });

  }

}
```

Important notes:

- `vx` randomization spreads particles horizontally
- `vy` negative values send particles slightly upward
- `life` determines how long they remain visible

Updating Particles Each Frame

Particles must be updated inside the draw loop.

Typical update sequence:

1 Update position

```
p.x += p.vx;
p.y += p.vy;
```

2 Apply slight gravity

```
p.vy += 0.1;
```

3️⃣ Decrease lifespan

```
p.life -= 1;
```

When `life` reaches zero, the particle should be removed.

Removing Expired Particles

To safely remove particles while looping through the array, iterate backwards.

Example:

```
for (let i = particles.length - 1; i >= 0; i--) {

  let p = particles[i];

  if (p.life <= 0) {
    particles.splice(i, 1);
  }

}
```

This avoids index shifting problems.

Drawing the Sparks ✨

Particles are usually drawn as small circles or rectangles.

Example:

```
circle(p.x, p.y, 3);
```

You can also fade particles as they age.

Example approach:

```
let alpha = map(p.life, 0, 20, 0, 255);
fill(255, 200, 80, alpha);
```

This makes the sparks fade out smoothly.

Triggering the Effect

The spark function should be called only when collisions or attacks occur.

Examples:

Wall collision:

```
if (player.colliding(wallL) || player.colliding(wallR)) {
  spawnSparks(player.x, player.y);
}
```

Attack event:

```
if (attackTriggered) {
  spawnSparks(player.x + offset, player.y);
}
```

This keeps particle generation tied directly to player interactions.

Implementation Tips ⚙️

Some useful practices when working with particle systems:

Keep particle counts moderate

Small bursts (8–15 particles) usually work well for sparks.

Avoid resetting the particle array

Particles should only be removed individually when their lifespan ends.

Ensure particles update every frame

If particles are created but not updated continuously, they will appear briefly and disappear.

Quick Implementation Summary

A simple spark particle system typically includes:

- an array to store particles

- a function that spawns particles
- an update loop that moves particles
- removal when lifespan reaches zero

This structure keeps the system predictable and easy to debug.

User:
I implemented the particle array and the sparks appear when the player hits the wall. The attack sparks also work now.

AI:

Good — that indicates the particle system is functioning correctly.

As long as:

- particles are updated every frame
- expired particles are removed from the array
- spawn events are tied to collision or attack triggers

the spark effect should remain stable while the sketch runs.

If additional particle types are added later, the same structure can be reused by adjusting parameters such as velocity ranges or lifespan values.

User:
The jump sound works correctly, but the landing sound seems slightly delayed. Sometimes the landing particle effect also triggers a bit later than expected. I want the landing feedback to happen immediately when the player touches the ground.

AI:

This type of timing issue usually happens because **landing is detected after the physics state has already changed**. In many platformer implementations, the player's vertical velocity (`vel.y`) is reset before the landing condition is evaluated, which can cause sound or visual effects to trigger one frame late.

To resolve this, it helps to detect **the exact moment when the player transitions from falling to grounded**.

Understanding the Timing Issue

In most physics systems, the following sequence happens each frame:

1. Apply gravity
2. Update velocity
3. Resolve collisions
4. Adjust position
5. Clamp or reset velocity

If landing detection is checked **after velocity is clamped to zero**, then the program cannot distinguish between:

- the moment of impact
- the frame after impact

This can cause landing effects to trigger later than expected.

Detecting a Landing Event

A reliable approach is to track the **previous frame's vertical state** and compare it to the current frame.

Typical pattern:

```
let previousVelocityY = 0;

function update() {

  let currentVelocityY = player.vel.y;

  if (previousVelocityY > 0 && player.colliding(ground)) {
    // landing detected
  }

  previousVelocityY = currentVelocityY;

}
```

Explanation:

- `previousVelocityY > 0` means the player was falling
- `colliding(ground)` confirms ground contact
- the combination indicates a landing event

Alternative Grounded-State Approach

Another method is to track a **grounded state transition**.

Example structure:

```
let wasGrounded = false;

function update() {

  let isGrounded = player.colliding(ground);

  if (!wasGrounded && isGrounded) {
    // landing event
  }

  wasGrounded = isGrounded;

}
```

Here the landing event triggers only when the grounded state changes.

Where to Trigger the Landing Effects

Once the landing moment is detected, the following feedback can be triggered inside that block:

- landing sound
- landing particles
- small camera shake

Example structure:

```
if (!wasGrounded && isGrounded) {

  playLandingSound();
  spawnLandingParticles();

}
```

The key idea is that **this code runs only once at the moment of impact**.

Common Causes of Delayed Landing Feedback

Some typical implementation details that can produce delays include:

Velocity reset before detection

```
if (player.colliding(ground)) {
  player.vel.y = 0;
}
```

If landing logic runs after this block, the falling state is lost.

Landing checks inside conditional blocks

Landing logic sometimes appears inside movement code or other state checks. If those blocks do not run every frame, landing events may not trigger immediately.

Particles triggered after position correction

If particles are spawned after position adjustments or physics corrections, they may appear slightly delayed relative to the impact.

Debugging Tip

To verify when landing is detected, temporarily log the event:

```
if (!wasGrounded && isGrounded) {
  console.log("Landing detected");
}
```

Watching the console output can confirm whether the landing detection happens at the correct frame.

Implementation Summary

To make landing feedback trigger immediately:

• track the previous frame state
• detect the transition from airborne to grounded
• trigger landing effects at that moment
• avoid placing landing logic after velocity resets

This approach helps ensure that landing sounds and particles occur at the exact moment of ground contact.

User:
I adjusted the landing detection logic and the landing feedback triggers more consistently now. The timing is better than before.

AI:

That indicates the landing event is now being detected at the correct moment in the update sequence.

As long as the landing condition is based on **state transitions rather than static checks**, the sound and visual effects should remain synchronized with the player's ground contact.

If additional interactions are added later, the same transition-based detection pattern can be reused for other events that depend on physics state changes.

User:
I want to add coins that the player can collect, a counter on the screen, and then a door that appears after all coins are collected. Also, I want an instruction overlay at the start that tells players the controls (A/D move, W jump, Space attack).

AI:

Got it ✅ — these features can be implemented cleanly using **three small systems**:

1. **Collectibles (coins)**
2. **Progress tracking + UI counter**
3. **Exit/door gating + win/reset**
4. **Instruction overlay (start-of-game UI)**

Below is an implementation approach that keeps the code organized and avoids common logic bugs.

---

1) Coins as Collectible Sprites

A straightforward structure is:

- create a `coins` array
- spawn coins in `setup()` (or a `resetCoins()` function)
- check overlaps each frame
- remove collected coins and increment a counter

Core variables:

- `let coins = [];`
- `let coinCount = 0;`

Example structure:

```
let coins = [];
let coinCount = 0;

function resetCoins() {

  for (let c of coins) c.remove();
  coins = [];

  coins.push(new Sprite(70, 100, 10, 10, "static"));
  coins.push(new Sprite(110, 100, 10, 10, "static"));
  coins.push(new Sprite(230, 65, 10, 10, "static"));

  for (let c of coins) {
    c.visible = false;       // hide collider box
    c.collider = "static";   // stable overlap target
  }

  coinCount = 0;
}
```

Why this is stable:

- Coins are stored in a single array
- Removal is controlled (not accidental)
- Respawning is centralized in one function

---

2) Coin Pickup Logic

Pickup detection is usually done using `overlaps()` (or an equivalent collision check) and removing the coin when collected.

Recommended pattern: iterate **backwards** when removing items:

```
for (let i = coins.length - 1; i >= 0; i--) {

  let c = coins[i];

  if (player.overlaps(c)) {

    coins.splice(i, 1);
    c.remove();

    coinCount++;

  }
```

```
}
```

This avoids index shifting issues when removing coins from the array.

Optional feedback hooks inside this block (only if you already decided to attach them):

- spawn particles
- play a pickup sound
- small screen shake

---

3) UI Counter for Coins

UI should be drawn with the camera turned off so it stays fixed on screen.

Typical pattern:

```
push();
camera.off();

fill(0);
textSize(12);
text("Coins: " + coinCount, 44, 18);

camera.on();
pop();
```

This ensures:

- UI stays on top
- UI doesn't move with the world
- UI remains readable

---

4) Door That Appears After All Coins Are Collected

This is easiest with a boolean flag:

- `exitActive = false` initially
- set it to `true` when the required coin count is reached
- only allow exit interaction when active

Variables:

- `let exitDoor;`

- `let exitActive = false;`

Activation rule example:

```
if (coinCount >= 3) {
  exitActive = true;
}
```

Exit interaction:

```
if (exitActive && player.overlaps(exitDoor)) {
  // trigger win state
}
```

Important implementation note ⚠️
Make sure this door-check logic is **not nested inside the coin loop**.
If it is inside the coin loop, and all coins are collected (so the array is empty), the loop will not run and the exit check will never execute.

So the correct structure is:

- coin collection loop runs
- exit activation check runs
- exit overlap check runs

---

5) Win State + Reset

A clean way is to introduce a simple timer:

- set `winTimer = 60` frames when the player reaches the door
- during winTimer: show a flash overlay / play win sound once
- when timer ends: reset the level

Example:

```
if (exitActive && player.overlaps(exitDoor) && winTimer === 0) {
  winTimer = 60;
}

if (winTimer > 0) {
  winTimer--;
  if (winTimer === 1) {
    resetLevel();
  }
}
```

Then `resetLevel()` can call:

- `resetCoins()`
- reset player position/velocity
- reset flags (`exitActive = false`, etc.)

---

6) Instruction Overlay at Start

A simple method is:

- `showHelp = true` at start
- hide it after:
  - a timer runs out, OR
  - the user presses any control key

Variables:

- `let showHelp = true;`
- `let helpTimer = 180;`

Overlay render:

```
function drawHelpOverlay() {

  push();
  camera.off();

  fill(255, 255, 255, 220);
  stroke(0);
  rect(26, 24, width - 52, 56);

  noStroke();
  fill(0);
  textSize(10);
  text("Controls:", 36, 38);
  text("A/D or \u2190/\u2192 : Move", 36, 52);
  text("W or \u2191 : Jump", 36, 64);
  text("Space : Attack", 36, 76);

  camera.on();
  pop();

}
```

Hide condition examples:

```
if (showHelp) {

  helpTimer--;

  if (helpTimer <= 0) showHelp = false;
```

```
  if (kb.presses("a") || kb.presses("d") || kb.presses("w") || kb.presses("
")) {
    showHelp = false;
  }

}
```

This keeps the overlay lightweight and purely informational.

Implementation Checklist ✅

To integrate these features smoothly, confirm you have:

- `coins` array + `coinCount`
- `resetCoins()` called in setup (or in resetLevel)
- backward loop pickup logic in draw
- UI draw using `camera.off()`
- `exitActive` flag logic outside the coin loop
- door overlap check outside the coin loop
- `winTimer` state + `resetLevel()`
- `showHelp/helpTimer` + overlay function

User:
I added coins, the counter, the door activation, and the instruction overlay. The door appears after collecting all coins and the UI shows the controls at the start.

AI:

Nice — that means your game loop is now complete and the interaction systems are working together:

- collectibles update progress
- progress unlocks the exit
- exit triggers a win state
- win state resets the experience
- instructions clarify the controls

As long as the exit logic stays outside the coin loop and the UI uses `camera.off()`, this setup should remain stable while you continue testing.